


# Inteligência Artificial

Problema de Satisfação de Restrições (CSP)

Setembro, 2026




# Motivação

**Questão**

Como encontrar uma atribuição de valores que satisfaça todas as restrições sem explorar cegamente todas as combinações?

# Problema de Satisfação de Restrições (CSP)

- Um problema de satisfação de restrições (CSP) é um problema matemático definido como um conjunto de objetos cujo estado deve satisfazer várias restrições e limitações.
- Um CSP é composto por:
  - Um conjunto de variáveis $X = \{X_1, X_2, \ldots, X_n\}$.
  - Um conjunto de domínios $D = \{D_1, D_2, \ldots, D_n\}$, onde cada $D_i$ é o conjunto de valores possíveis para a variável $X_i$.
  - Um conjunto de restrições $C = \{C_1, C_2, \ldots, C_m\}$, onde cada restrição $C_j$ especifica os valores permitidos para um subconjunto de variáveis.
- O objetivo é encontrar uma atribuição de valores às variáveis que satisfaça todas as restrições.

# Problema de Satisfação de Restrições (CSP)

- CSPs lidam com **atribuições** de valores a variáveis, enquanto problemas de otimização buscam maximizar ou minimizar uma função objetivo.
- Uma atribuição é uma função que mapeia cada variável a um valor em seu domínio.
- Uma atribuição é **consistente** se não viola nenhuma restrição.
- Uma atribuição é **completa** se todas as variáveis têm valores atribuídos.
- Uma **solução** para um CSP é uma atribuição que é tanto consistente quanto completa.

# Exemplos de CSPs

- Problema das N-Rainhas

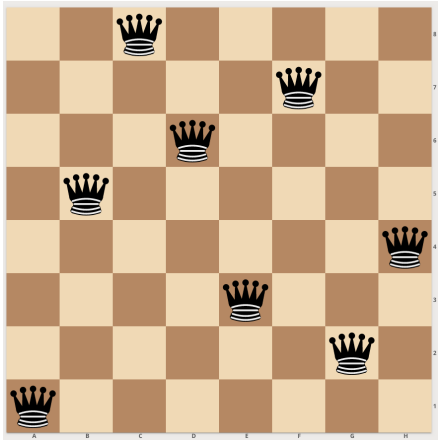


Figure: Problema das N-Rainhas.

# Exemplos de CSPs

- Problema das N-Rainhas
- Sudoku

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 9 | 8 | 5 | 4 | | 1 | | | |
| | | | | 3 | | | | |
| 1 | | 6 | | | | | | |
| | | | 5 | | | | | |
| 4 | | 2 | | | 9 | | | 3 |
| | 9 | | | 6 | 3 | 4 | | |
| | 6 | | | 1 | | | | |
| | | | 3 | | 6 | | | 5 |
| 2 | | | | 8 | | | | 1 |

Figure: Problema do Sudoku.

# Exemplos de CSPs

- Problema das N-Rainhas
- Sudoku
- Coloração de Grafos

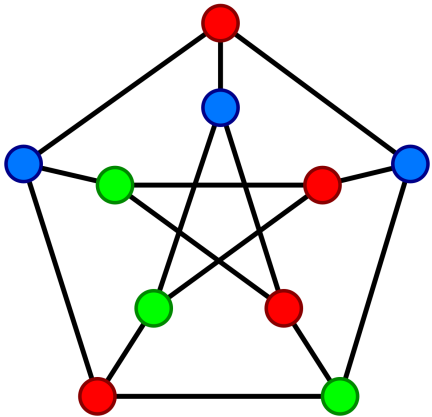


Figure: Problema de Coloração de Grafos.

# Exemplos de CSPs

- Problema das N-Rainhas
- Sudoku
- Coloração de Grafos
- Problema do Caixeiro Viajante (TSP)

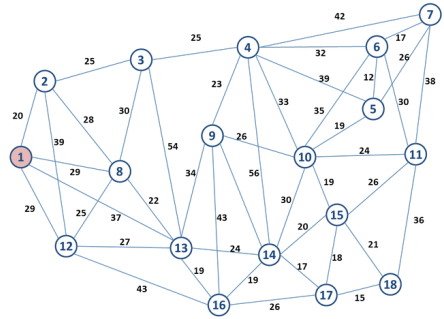



Figure: Problema do Caixeiro Viajante (TSP).

# Problema de Coloração de Grafos

- Dado um grafo $G = (V, E)$, onde $V$ é o conjunto de vértices e $E$ é o conjunto de arestas, o objetivo é atribuir uma cor a cada vértice de modo que nenhuma aresta conecte dois vértices da mesma cor.
- Variáveis: Cada vértice $v_i \in V$ é uma variável.
- Domínios: O domínio de cada variável é o conjunto de cores disponíveis.
- Restrições: Para cada aresta $(v_i, v_j) \in E$, a restrição é que as cores atribuídas a $v_i$ e $v_j$ devem ser diferentes.

# Problema de Coloração de Grafos

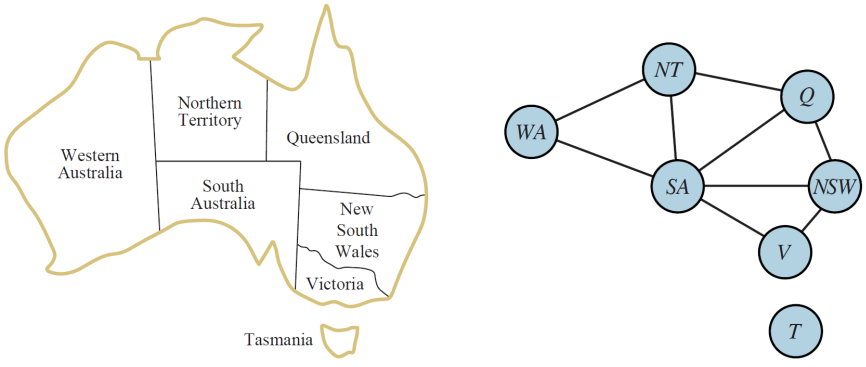



Figure: Exemplo de Coloração de Grafos.

# Problema de Coloração de Grafos

- Variáveis:
  $X = \{WA, NT, SA, Q, NSW, V, T\}$

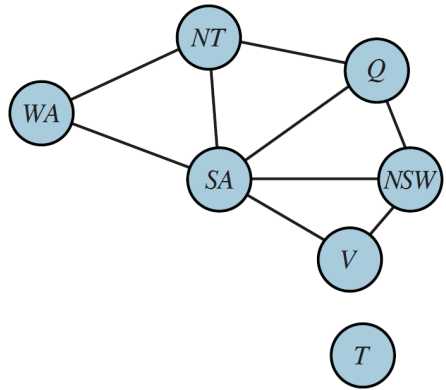



Figure: Exemplo de Coloração de Grafos.

# Problema de Coloração de Grafos

- Variáveis:
  $X = \{WA, NT, SA, Q, NSW, V, T\}$
- Domínios:
  $D = \{Vermelho, Verde, Azul\}$

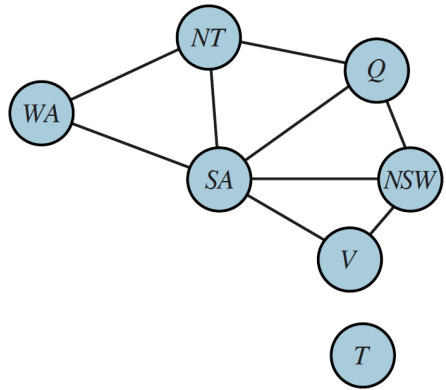



Figure: Exemplo de Coloração de Grafos.

# Problema de Coloração de Grafos

- Variáveis:
  $X = \{WA, NT, SA, Q, NSW, V, T\}$
- Domínios:
  $D = \{Vermelho, Verde, Azul\}$
- Restrições:
  - $WA \neq NT$
  - $WA \neq SA$
  - $NT \neq SA$
  - $NT \neq Q$
  - $SA \neq Q$
  - $SA \neq NSW$
  - $SA \neq V$
  - $Q \neq NSW$
  - $NSW \neq V$

Restrição binária

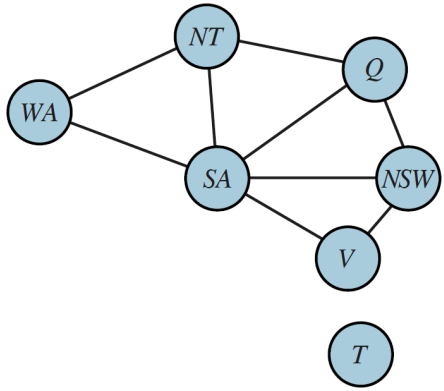


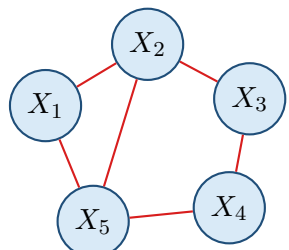

Figure: Exemplo de Coloração de Grafos.

# Grafo de restrições

- Cada nó representa uma variável.
- Cada aresta representa uma restrição entre duas variáveis.

Esse grafo é central porque influencia:

- **Degree Heuristic**;
- **Forward Checking**;
- **AC-3**;
- **estrutura do problema**.



# Tipos de restrições

- **Unária**: envolve uma variável.

$$X \neq \text{vermelho}$$

- **Binária**: envolve duas variáveis.

$$X \neq Y$$

- **Global**: envolve várias variáveis.

$$AllDiff(X_1, \ldots, X_n)$$

## Exemplos

- Sudoku: muitas restrições do tipo `AllDiff`.
- N-Rainhas: restrições entre pares de rainhas.
- Escalonamento: restrições temporais, de recurso, precedência etc.

# Backtracking

- Backtracking é uma técnica de busca que constrói soluções incrementais, abandonando soluções parciais assim que determina que elas não podem ser completadas para formar uma solução válida.
- No contexto de CSPs, o backtracking tenta atribuir valores às variáveis uma de cada vez, verificando as restrições a cada passo.
- Se uma atribuição leva a um conflito (ou seja, viola uma restrição), o algoritmo retrocede (backtracks) para a variável anterior e tenta um valor diferente.
- O processo continua até que todas as variáveis sejam atribuídas consistentemente ou até que todas as possibilidades sejam esgotadas.

# Backtracking - Algoritmo

- Função Backtrack(atribuição):
  1. Se a atribuição é completa, retorne a atribuição.
  2. Se não, selecione uma variável não atribuída $X$.
  3. Para cada valor $v$ no domínio de $X$:
     - Se a atribuição de $X = v$ é consistente com as restrições: adicione $X = v$ à atribuição e chame recursivamente Backtrack(atribuição). Se a chamada recursiva retornar uma solução, retorne essa solução; caso contrário, remova $X = v$ da atribuição (backtrack) e continue.
  4. Retorne falha (nenhum valor levou a uma solução).

# Backtracking - Pseudocódigo

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences ← INFERENCE(csp, var, value)
      if inferences ≠ failure then
        add inferences to assignment
        result ← BACKTRACK(assignment, csp)
        if result ≠ failure then
          return result
    remove {var = value} and inferences from assignment
  return failure
```

Figure: Pseudocódigo do Algoritmo de Backtracking.

# Backtracking - Exemplo





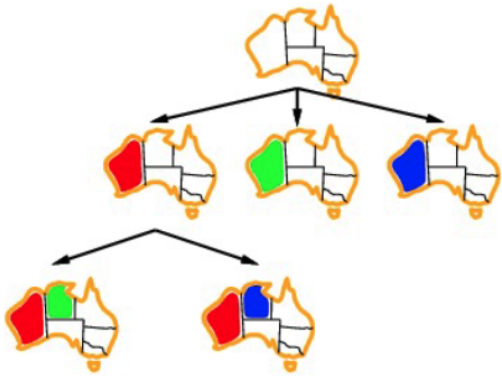

Figure: Exemplo de Backtracking.

# Backtracking - Exemplo





# Backtracking - Exemplo





# Backtracking - Exemplo

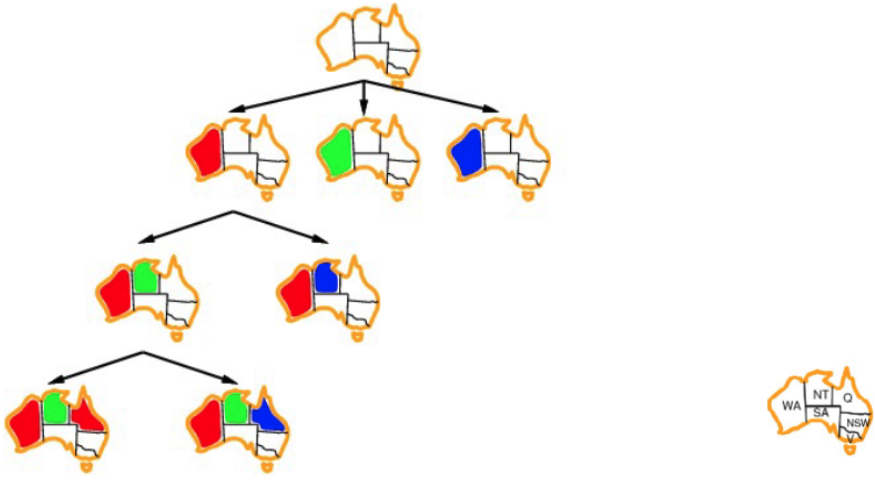



Figure: Exemplo de Backtracking.

# Backtracking - Ordenação de variáveis e valores

- A escolha da ordem em que as variáveis são atribuídas pode ter um impacto significativo na eficiência do algoritmo de backtracking.
- Heurísticas comuns incluem:
  - **Minimum Remaining Values (MRV)**: Escolha a variável com o menor número de valores restantes em seu domínio.
  - **Degree Heuristic**: Escolha a variável que está envolvida no maior número de restrições com outras variáveis não atribuídas.
- A ordem em que os valores são tentados para uma variável também pode afetar o desempenho. Uma heurística comum é:
  - **Least Constraining Value**: Tente primeiro o valor que restringe menos os valores disponíveis para as outras variáveis.

# MRV e Degree Heuristic

## MRV

Escolha a variável com o **menor número de valores legais restantes**.

Mais restrita primeiro

## Degree Heuristic

Entre variáveis empatadas em MRV, escolha a que participa do **maior número de restrições** com variáveis ainda não atribuídas.

Mais conectada primeiro

## Estratégia combinada

$$\text{Escolha variável} = \text{MRV} \xrightarrow{\text{empate}} \text{Degree}$$

# Exemplo de escolha de variável

Suponha os domínios atuais:

| Variável | Domínio legal | Grau restante |
|---|---|---|
| $X_1$ | $\{1, 2\}$ | 2 |
| $X_2$ | $\{1\}$ | 3 |
| $X_3$ | $\{1, 2, 3\}$ | 4 |
| $X_4$ | $\{2\}$ | 1 |

# Exemplo de escolha de variável

Suponha os domínios atuais:

| Variável | Domínio legal | Grau restante |
|---|---|---|
| $X_1$ | $\{1, 2\}$ | 2 |
| $X_2$ | $\{1\}$ | 3 |
| $X_3$ | $\{1, 2, 3\}$ | 4 |
| $X_4$ | $\{2\}$ | 1 |

- Pelo **MRV**, candidatas:

# Exemplo de escolha de variável

Suponha os domínios atuais:

| Variável | Domínio legal | Grau restante |
|---|---|---|
| $X_1$ | $\{1, 2\}$ | 2 |
| $X_2$ | $\{1\}$ | 3 |
| $X_3$ | $\{1, 2, 3\}$ | 4 |
| $X_4$ | $\{2\}$ | 1 |

- Pelo **MRV**, candidatas:

$$X_2,\ X_4$$

- Empate?

# Exemplo de escolha de variável

Suponha os domínios atuais:

| Variável | Domínio legal | Grau restante |
|---|---|---|
| $X_1$ | $\{1, 2\}$ | 2 |
| $X_2$ | $\{1\}$ | 3 |
| $X_3$ | $\{1, 2, 3\}$ | 4 |
| $X_4$ | $\{2\}$ | 1 |

- Pelo **MRV**, candidatas:

$$X_2,\ X_4$$

- Empate? Usamos **Degree Heuristic**.

# Exemplo de escolha de variável

Suponha os domínios atuais:

| Variável | Domínio legal | Grau restante |
|---|---|---|
| $X_1$ | $\{1, 2\}$ | 2 |
| $X_2$ | $\{1\}$ | 3 |
| $X_3$ | $\{1, 2, 3\}$ | 4 |
| $X_4$ | $\{2\}$ | 1 |

- Pelo **MRV**, candidatas:

$$X_2,\ X_4$$

- Empate? Usamos **Degree Heuristic**.
- Como $X_2$ tem maior grau restante:

$\boxed{X_2 \text{ é escolhida}}$

# Least Constraining Value (LCV)

## Ideia

Ao escolher um valor para uma variável, tente primeiro o valor que remove o **menor número** de opções das variáveis vizinhas ainda não atribuídas.

Se $X$ pode receber:

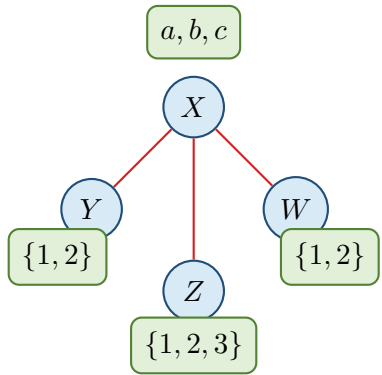
$$\{a, b, c\}$$

e:

- $a$ elimina 4 valores vizinhos;
- $b$ elimina 1 valor vizinho;
- $c$ elimina 3 valores vizinhos;

então a ordem LCV é:

$$\boxed{b,\ c,\ a}$$



# Backtracking - Inferência - Foward Checking

- O Foward Checking é uma técnica de inferência usada em conjunto com o backtracking para reduzir o espaço de busca.
- Sempre que uma variável é atribuída, o Foward Checking verifica todas as variáveis não atribuídas que são diretamente conectadas à variável recém-atribuída por uma restrição.
- Para cada uma dessas variáveis, o Foward Checking remove do seu domínio qualquer valor que seja inconsistente com a atribuição atual.
- Se o domínio de qualquer variável não atribuída se tornar vazio, o algoritmo retrocede imediatamente, pois a atribuição atual não pode levar a uma solução válida.

# Backtracking - Inferência - Foward Checking

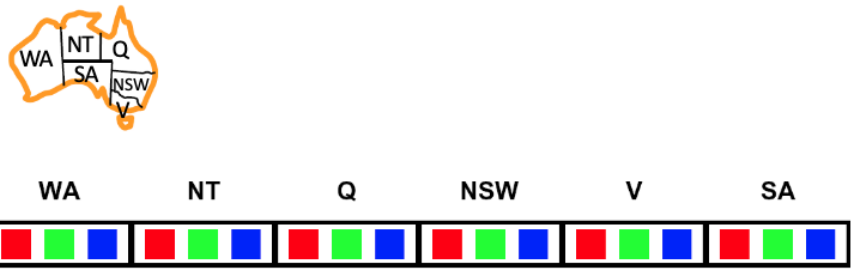



Figure: Pseudocódigo do Foward Checking.

# Backtracking - Inferência - Foward Checking

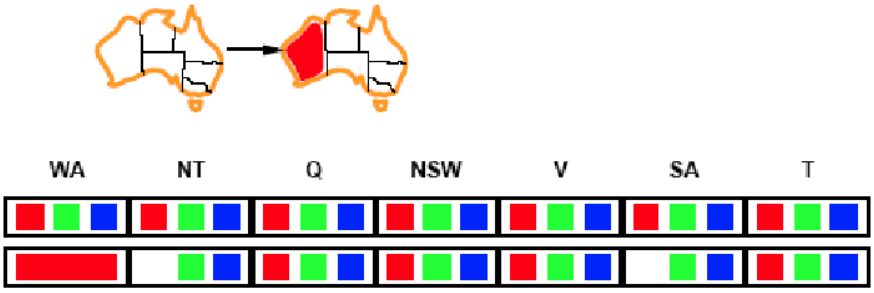



Figure: Pseudocódigo do Foward Checking.

- Efeitos
  - NT não pode mais ser vermelho
  - SA não pode mais ser vermelho

# Backtracking - Inferência - Foward Checking

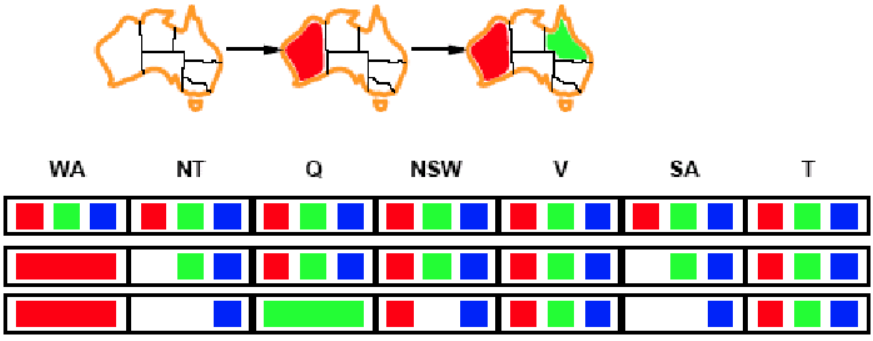



Figure: Pseudocódigo do Foward Checking.

- Efeitos
  - NT não pode mais ser verde
  - NSW não pode mais ser verde
  - SA não pode mais ser verde

# Backtracking - Inferência - Foward Checking

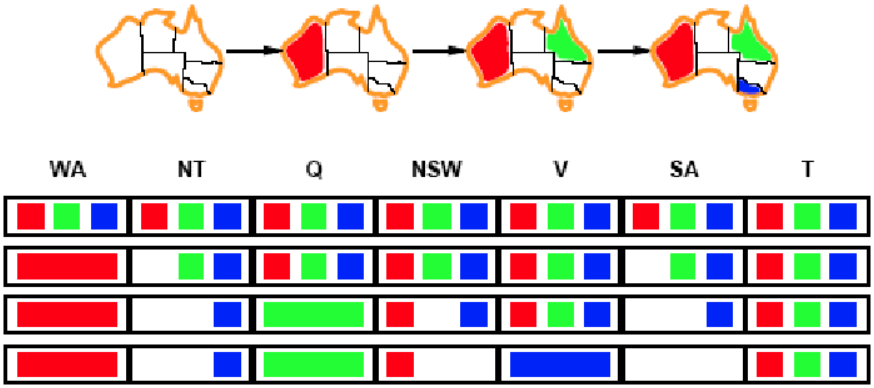



Figure: Pseudocódigo do Foward Checking.

- Efeitos
  - Domínio legal de SA é vazio
  - Detecção parcial inconsistente ⇒ backtrack

# Preprocessamento - Consistência de Arcos

- A consistência de arcos é uma técnica de preprocessamento que pode ser aplicada antes do início do backtracking para reduzir os domínios das variáveis.
- Uma variável $X_i$ é consistente com outra variável $X_j$ se, para cada valor em $D_i$, existe pelo menos um valor em $D_j$ que não viola a restrição entre $X_i$ e $X_j$.

$$\boxed{\forall x \in D_i,\ \exists y \in D_j \text{ tal que } (x, y) \text{ satisfaz a restrição}}$$

**Consequência**

Se um valor de $D_i$ não tem suporte em $D_j$, esse valor pode ser removido.

# Preprocessamento - Consistência de Arcos

- O algoritmo AC-3 (Arc Consistency Algorithm 3) é um método popular para garantir a consistência de arcos em um CSP.
- O AC-3 itera sobre todas as arestas (pares de variáveis conectadas por uma restrição) e remove valores dos domínios que não são consistentes.
- O algoritmo AC-3 funciona da seguinte maneira:
  1. Inicialize uma fila com todas as arestas do grafo de restrições.
  2. Enquanto a fila não estiver vazia:
     - Remova uma aresta $(X_i, X_j)$ da fila.
     - Revise o domínio de $X_i$ para garantir que cada valor em $D_i$ tenha um valor correspondente em $D_j$ que satisfaça a restrição.
     - Se algum valor for removido de $D_i$, adicione todas as arestas $(X_k, X_i)$ de volta à fila, onde $X_k$ é qualquer variável conectada a $X_i$ por uma restrição.

# Preprocessamento - Consistência de Arcos - Exemplo

- Considere um mapa com três regiões interligadas: Região A, Região B e Região C.
  - Restrições: Regiões adjacentes não podem ter a mesma cor.
  - A é adjacente a B e C.
  - B é adjacente a A e C.
  - Domínios iniciais: $D_A = \{Vermelho, Verde\}$, $D_B = \{Vermelho, Azul\}$, $D_C = \{Verde\}$.

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.
  - Arco: (B, A) - Verificar: Cada valor em Dom(B) deve ter um correspondente em Dom(A).

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.
  - Arco: (B, A) - Verificar: Cada valor em Dom(B) deve ter um correspondente em Dom(A).
  - Dom(B) permanece o mesmo.

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.
  - Arco: (B, A) - Verificar: Cada valor em Dom(B) deve ter um correspondente em Dom(A).
  - Dom(B) permanece o mesmo.
  - Arco: (A, C) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(C).

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F $= [(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.
  - Arco: (B, A) - Verificar: Cada valor em Dom(B) deve ter um correspondente em Dom(A).
  - Dom(B) permanece o mesmo.
  - Arco: (A, C) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(C).
  - Verde é removido de Dom(A).

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F = $[(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.
  - Arco: (B, A) - Verificar: Cada valor em Dom(B) deve ter um correspondente em Dom(A).
  - Dom(B) permanece o mesmo.
  - Arco: (A, C) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(C).
  - Verde é removido de Dom(A).
- Domínios: $D_A = \{Vermelho\}$, $D_B = \{Vermelho, Azul\}$, $D_C = \{Verde\}$.

# Preprocessamento - Consistência de Arcos - Exemplo

- O AC-3 adiciona todos os arcos bidirecionais adjacentes à Fila de Trabalho (F).
  - F = $[(A, B), (B, A), (A, C), (C, A), (B, C), (C, B)]$
- O algoritmo processa a fila até que ela esteja vazia.
  - Arco: (A, B) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(B).
  - Dom(A) permanece o mesmo.
  - Arco: (B, A) - Verificar: Cada valor em Dom(B) deve ter um correspondente em Dom(A).
  - Dom(B) permanece o mesmo.
  - Arco: (A, C) - Verificar: Cada valor em Dom(A) deve ter um correspondente em Dom(C).
  - Verde é removido de Dom(A).
- Domínios: $D_A = \{Vermelho\}$, $D_B = \{Vermelho, Azul\}$, $D_C = \{Verde\}$.
- Final: $D_A = \{Vermelho\}$, $D_B = \{Azul\}$, $D_C = \{Verde\}$.

# Forward Checking vs AC-3

| Técnica | O que faz? | Vantagem | Limitação |
|---|---|---|---|
| Forward Checking | Após atribuir $X$, poda domínios dos vizinhos | Simples e barato | Propagação limitada |
| AC-3 | Impõe consistência de arcos em toda a rede | Propaga mais e detecta falhas antes | Mais custo por passo |

Mais inferência $\Rightarrow$ menos busca, mas maior custo local

# Resumo

1. Fundamentos do CSP
   - **Definição**: Um Problema de Satisfação de Restrições (CSP) é definido por três componentes: **Variáveis** (os elementos a serem definidos), **Domínios** (os valores possíveis) e **Restrições** (as regras que limitam as atribuições).
   - **Objetivo**: Encontrar uma atribuição completa de valores para todas as variáveis que não viole nenhuma restrição. Não há custo a ser minimizado; a busca é binária: existe solução ou não existe.
   - **Exemplos**: Problemas clássicos como Coloração de Mapas, Oito Rainhas e Sudoku são modelados como CSPs.

# Resumo

2. Retrocesso (Backtracking)
   - **Mecanismo de Busca**: O Retrocesso é o algoritmo de busca fundamental para CSPs. Ele funciona como uma Busca em Profundidade (DFS), tentando atribuir um valor a uma variável por vez.
   - **Violação e Retorno**: Se uma atribuição viola uma restrição, o algoritmo imediatamente retrocede (volta) para a variável anterior para tentar um valor diferente. Isso evita que a busca explore caminhos que já são inviáveis.

# Resumo

3. Aprimorando a Eficiência (Heurísticas)
   - **MRV (Minimum Remaining Values)**: Escolher a **variável mais restrita** (com o menor número de valores legais restantes) primeiro. O princípio é **falhar cedo é melhor** — se essa variável não puder ser resolvida, é melhor descobrir isso o quanto antes.
   - **LCV (Least Constraining Value)**: Ao escolher o valor para uma variável, tentar o **valor que restringe menos** as opções futuras das variáveis vizinhas. O princípio é **deixar mais opções abertas para o futuro**.

# Resumo

4. Propagação de Restrições
   - **Função**: A propagação de restrições é uma técnica que reduz os domínios das variáveis, eliminando valores que são comprovadamente inconsistentes com as restrições, antes ou durante a busca.
   - **Forward Checking**: A técnica mais comum. Sempre que uma variável é atribuída a um valor, o Forward Checking remove todos os valores conflitantes dos domínios das variáveis ainda não atribuídas. Isso permite que o sistema **detecte falhas mais cedo** na árvore de busca.

# Atividade

1. Qual é a principal diferença entre a resolução de um Problema de Satisfação de Restrições (CSP) e a Busca em Largura (BFS) ou A*?
   a) CSP sempre utiliza heurísticas, enquanto BFS e A* não usam.
   b) O CSP busca o caminho de menor custo, enquanto BFS e A* buscam apenas uma solução.
   c) CSP foca em encontrar um estado que satisfaça todas as regras, sem otimizar o custo.
   d) BFS e A* garantem a otimalidade, e o CSP sempre encontra a solução mais rápida.

# Atividade

2. O que representa o Domínio (D) no modelo formal de um CSP?
   a) O conjunto de todas as variáveis do problema.
   b) O conjunto de todos os valores possíveis que podem ser atribuídos a uma variável.
   c) O conjunto de todas as restrições que devem ser satisfeitas.
   d) O conjunto de todas as soluções possíveis para o problema.

# Atividade

3. Qual das seguintes heurísticas é usada no retrocesso para escolher a variável a ser atribuída em seguida, com o objetivo de falhar rapidamente?
   a) Heurística de Valor Menos Restritivo (LCV)
   b) Heurística do Primeiro Sucesso
   c) Heurística de Valores Mínimos Restantes (MRV)
   d) Busca em Largura (BFS)

# Atividade

4. A principal função do algoritmo de Retrocesso (Backtracking) é:
   a) Eliminar todos os valores do domínio das variáveis não atribuídas no início da busca.
   b) Voltar para o nó anterior e tentar um caminho diferente quando uma restrição é violada.
   c) Avançar de forma aleatória na árvore de busca para evitar loops.
   d) Garantir que a solução encontrada seja a de menor custo total.

# Atividade

5. O que a heurística LCV (Least Constraining Value) tenta alcançar ao escolher um valor para uma variável?
   a) Garantir que o valor escolhido satisfaça todas as restrições.
   b) Escolher o valor que minimiza o número de atribuições necessárias.
   c) Atribuir um valor que deixa o maior número de opções disponíveis para as variáveis vizinhas não atribuídas.
   d) Garantir que o algoritmo retroceda o mínimo possível.

# Atividade

6. A técnica de Forward Checking é uma forma simples de propagação de restrições. Qual é o seu principal benefício?
   a) Permitir que o agente detecte falhas mais cedo (na atribuição), reduzindo a profundidade de busca.
   b) Armazenar todo o histórico de atribuições para fins de aprendizado de máquina.
   c) Garantir que a solução final seja a de menor custo.
   d) Eliminar valores conflitantes das variáveis já atribuídas.

# Atividade

**7.**

$$X = \{A, B, C\}$$

$$D_A = \{1, 2\}, \quad D_B = \{1, 2\}, \quad D_C = \{1, 2, 3\}$$

Restrições:

$$A \neq B, \quad B \neq C$$

Responda:

a) Qual variável seria escolhida por **MRV**?

b) Se houver empate, a **Degree Heuristic** decide algo?

c) Se $B = 1$, como ficam os domínios após **Forward Checking**?

# Atividade

8. Considere os domínios:

$$D_A = \{\text{Vermelho, Verde}\}, \quad D_B = \{\text{Vermelho, Azul}\}, \quad D_C = \{\text{Verde}\}$$

Restrições:

$$A \neq B, \quad A \neq C, \quad B \neq C$$

a) Execute manualmente o **AC-3**.
b) Quais valores são removidos?
c) Qual é o estado final da fila de arcos?